Software Requirements Specification

A Mobile Application for Facility Management

Team
Keda keda

Members
Mostafa amer 13001482
Youssef tamer 13007100
Mohamed haitham 13007066
Nariman sherif 13006930
Habiba elwi 13001343
Hla ezzeldin 13001687
Vania abdelghafar 13007464
Israa hagag 13004569

Tutorial Details
[2] | [Momen]

1. Introduction

CampusCare is a mobile application designed to improve facility management within a university campus. The system connects students and staff with the Facility Management Team through a centralized digital platform that enables efficient reporting and handling of maintenance issues.
Many campus problems such as damaged furniture, electrical faults, or plumbing issues often go unreported or are communicated through unclear channels. This leads to delayed maintenance, inefficient workflows, and poor communication between users and administrators. CampusCare addresses these challenges by providing a structured and accessible system where users can report issues, track their status, and receive updates in real time.
This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of the CampusCare system. It is intended for instructors, teaching assistants, developers, testers, and stakeholders involved in the project.

1.1 Product Vision & Scope

The vision of CampusCare is to create a smart and efficient campus environment by digitalizing the process of reporting and managing maintenance issues. The application aims to simplify communication between the campus community and the facility management team, ensuring faster response times and improved transparency.

The system consists of two primary user roles. Community members, including students and staff, can report issues by submitting a ticket that includes a photo, location, category, and description. Facility managers or administrators can access a dashboard where they receive notifications of new issues, review submitted tickets, update their status, and manage the maintenance workflow until resolution.

The scope of the system focuses on maintenance-related problems within the campus, such as electrical, plumbing, and infrastructure issues. The system aims to improve reporting efficiency, reduce response time, and enhance overall communication between users and administrators.

1.2 Definitions and Acronyms

SRS (Software Requirements Specification): A document that describes the system's features, requirements, and constraints.
Ticket: A digital record representing a maintenance issue reported by a user.
RBAC (Role-Based Access Control): A system where access permissions are assigned based on user roles.
MVP (Minimum Viable Product): The initial version of the system with only essential features.

2. Functional Requirements

Community Member: A student or staff member who uses the system to report campus issues.

Admin / Facility Manager: A user responsible for monitoring, managing, and resolving reported issues.

2.1 User stories:

1. Community Member

US-CM-01:
As a Community Member, I should be able to create an account so that I can log in to the application.

US-CM-02:
As a Community Member, I should be able to log in using my email and password so that I can access the application

US-CM-03:
As a Community Member, I should be able to submit an issue with its details (location, photo, and category) so that the problem can be reported accurately.

US-CM-04:
As a Community Member, I should be able to add a description to the issue so that the worker understands the problem clearly.

US-CM-05:
As a Community Member, I should be able to view my submitted issues so that I can track their status.

US-CM-06:
As a Community Member, I should be able to log out so that my data remains secure on shared devices.

US-CM-07
As a Community Member, I should be able to view the status of my issue (Pending, In Progress, Resolved) so that I know the progress of my request.

2. Facility Manager

US-FM-01:
As a Facility Manager, I should be able to create an account so that I can log in to the application.

US-FM-02:
As a Facility Manager, I should be able to log in using my email and password so that I can access the application.

US-FM-03:
As a Facility Manager, I should be able to log out so that my data remains secure on shared devices.

US-FM-04:
As a Facility Manager, I should be able to view all submitted issues so that I can assign them to workers.

US-FM-05:
As a Facility Manager, I should be able to update an issue’s status so that the system reflects the current state of the issue.

US-FM-06

As a Facility Manager, I should be able to assign an issue to a worker so that the worker can start fixing it.

US-FM-07
As a Facility Manager, I should be able to close an issue after it is resolved so that the system reflects that the issue is completed.

3. Worker
US-W-01:
As a Worker, I should be able to create an account so that I can log in to the application.

US-W-02:
As a Worker, I should be able to log in using my email and password so that I can access the application.

US-W-03:
As a Worker, I should be able to log out so that my data remains secure on shared devices.

US-W-04:
As a Worker, I should be able to view my assigned issues so that I can start working on them.

US-W-05:
As a Worker, I should be able to comment on an issue to indicate that the work is completed so that the facility manager can update its status.

US-W-06
As a Worker, I should be able to mark an issue as “In Progress” so that the facility manager and community member know that it is being worked on.

US-W-07
As a Worker, I should be able to upload a photo after fixing the issue so that the facility manager can verify the work.

4. System Admin (optional for Now):
US-SA-01:
As a System Admin, I should be able to log in so that I can access the application.

US-SA-02:
As a System Admin, I should be able to log out so that my data remains secure on shared devices.

US-SA-03:
As a System Admin, I should be able to activate or deactivate any account so that I can ensure all users follow the system rules.

US-SA-04
As a System Admin, I should be able to view all registered users so that I can manage the system users.

Functional Requirements

1. **Community Member Functional Requirements**

FR-CM-01: Account Creation
The system shall allow a Community Member to create an account by providing required information (e.g., name, email, and password).
FR-CM-02: User Authentication
The system shall allow a Community Member to log in using a valid email and password.
FR-CM-03: Issue Submission
The system shall allow a Community Member to submit an issue including the following details:

- Location
- Photo
- Category
- Description

FR-CM-04: Issue Description
The system shall allow a Community Member to add a textual description when submitting an issue.
FR-CM-05: View Submitted Issues
The system shall allow a Community Member to view a list of issues they have submitted.
FR-CM-06: Logout
The system shall allow a Community Member to log out of the application.
FR-CM-07: View Issue Status
The system shall display the current status of an issue to the Community Member.
Possible statuses include:

- Pending
- In Progress
- Resolved

2. Facility Manager Functional Requirements
FR-FM-01: Account Creation
The system shall allow a Facility Manager to create an account.
FR-FM-02: User Authentication
The system shall allow a Facility Manager to log in using a valid email and password.
FR-FM-03: Logout
The system shall allow a Facility Manager to log out of the application.
FR-FM-04: View All Issues
The system shall allow a Facility Manager to view all submitted issues in the system.
FR-FM-05: Update Issue Status
The system shall allow a Facility Manager to update the status of an issue.
FR-FM-06: Assign Issue to Worker
The system shall allow a Facility Manager to assign an issue to a specific worker.
FR-FM-07: Close Issue
The system shall allow a Facility Manager to close an issue once it has been resolved.

---

3. Worker Functional Requirements

FR-W-01: Account Creation
The system shall allow a Worker to create an account.
FR-W-02: User Authentication
The system shall allow a Worker to log in using a valid email and password.
FR-W-03: Logout
The system shall allow a Worker to log out of the application.
FR-W-04: View Assigned Issues
The system shall allow a Worker to view issues assigned to them.
FR-W-05: Comment on Issue
The system shall allow a Worker to add comments to an issue.
FR-W-06: Update Issue Status to In Progress
The system shall allow a Worker to mark an assigned issue as **In Progress**.
FR-W-07: Upload Completion Photo
The system shall allow a Worker to upload a photo after completing the work on an issue.

---

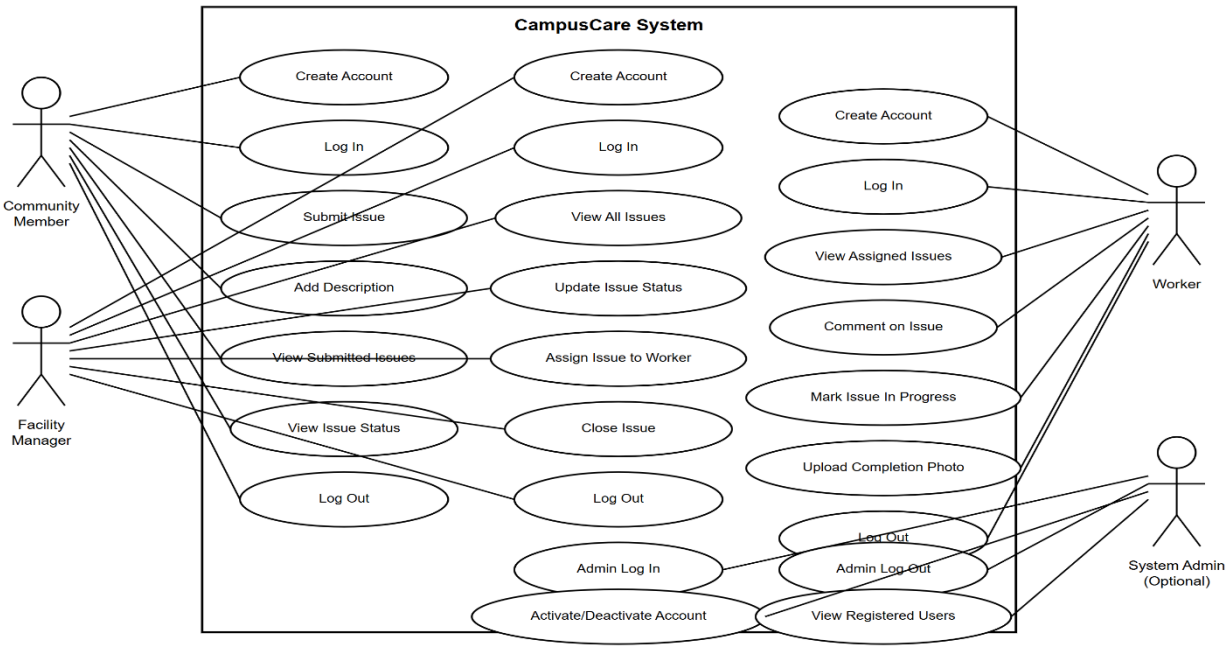
4. System Admin Functional Requirements (Optional)
FR-SA-01: Admin Authentication
The system shall allow a System Admin to log in.
FR-SA-02: Logout
The system shall allow a System Admin to log out.
FR-SA-03: Account Management
The system shall allow a System Admin to activate or deactivate user accounts.
FR-SA-04: View Registered Users
The system shall allow a System Admin to view all registered users in the system.

Product BackLog

Backlog ID	User Story ID	Priority
Core System Features		
PB-01	US-CM-01	High
PB-02	US-CM-02	High
PB-03	US-CM-03	High
PB-04	US-CM-04	High
PB-05	US-CM-05	High
PB-06	US-CM-07	High
PB-07	US-FM-01	High
PB-08	US-FM-02	High
PB-09	US-W-01	High
PB-10	US-W-02	High

Issue Management Features		
PB-11	US-FM-04	High
PB-12	US-FM-06	High
PB-13	US-W-04	High
PB-14	US-W-06	High
PB-15	US-W-05	Medium
PB-16	US-W-07	Medium
PB-17	US-FM-05	High
PB-18	US-FM-07	High
Authentication & Security		
PB-19	US-CM-06	Medium
PB-20	US-FM-03	Medium
PB-21	US-W-03	Medium
System Administration (Optional)		
PB-22	US-SA-01	Low
PB-23	US-SA-02	Low
PB-24	US-SA-03	Low
PB-25	US-SA-04	Low

2.2 UML use case Diagram



3. Non-Functional requirements

- **NFR-01 (Performance):** The system shall maintain response and loading times under three seconds under normal conditions.
- **NFR-02 (Scalability):** The system shall support at least 500 simultaneous users without performance degradation.
- **NFR-03 (Security):** The system shall implement secure authentication, input validation, and protection against unauthorized access.
- **NFR-04 (Usability):** The system shall provide a simple and intuitive interface that allows users to submit a ticket within one minute.
- **NFR-05 (Reliability):** The system shall ensure stable performance and reliable image uploads even under unstable network conditions.
- **NFR-06 (Availability):** The system shall be accessible at all times except during scheduled maintenance.
- **NFR-07 (Backup and Recovery):** The system shall perform regular backups and support data recovery in case of failure.
- **NFR-08 (Maintainability):** The system shall be designed with modular components to allow easy updates and maintenance.
- **NFR-09 (Compatibility):** The mobile application shall support common Android and iOS devices.
- **NFR-10 (Data Integrity):** The system shall ensure accuracy and consistency of stored data.

4. Constraints

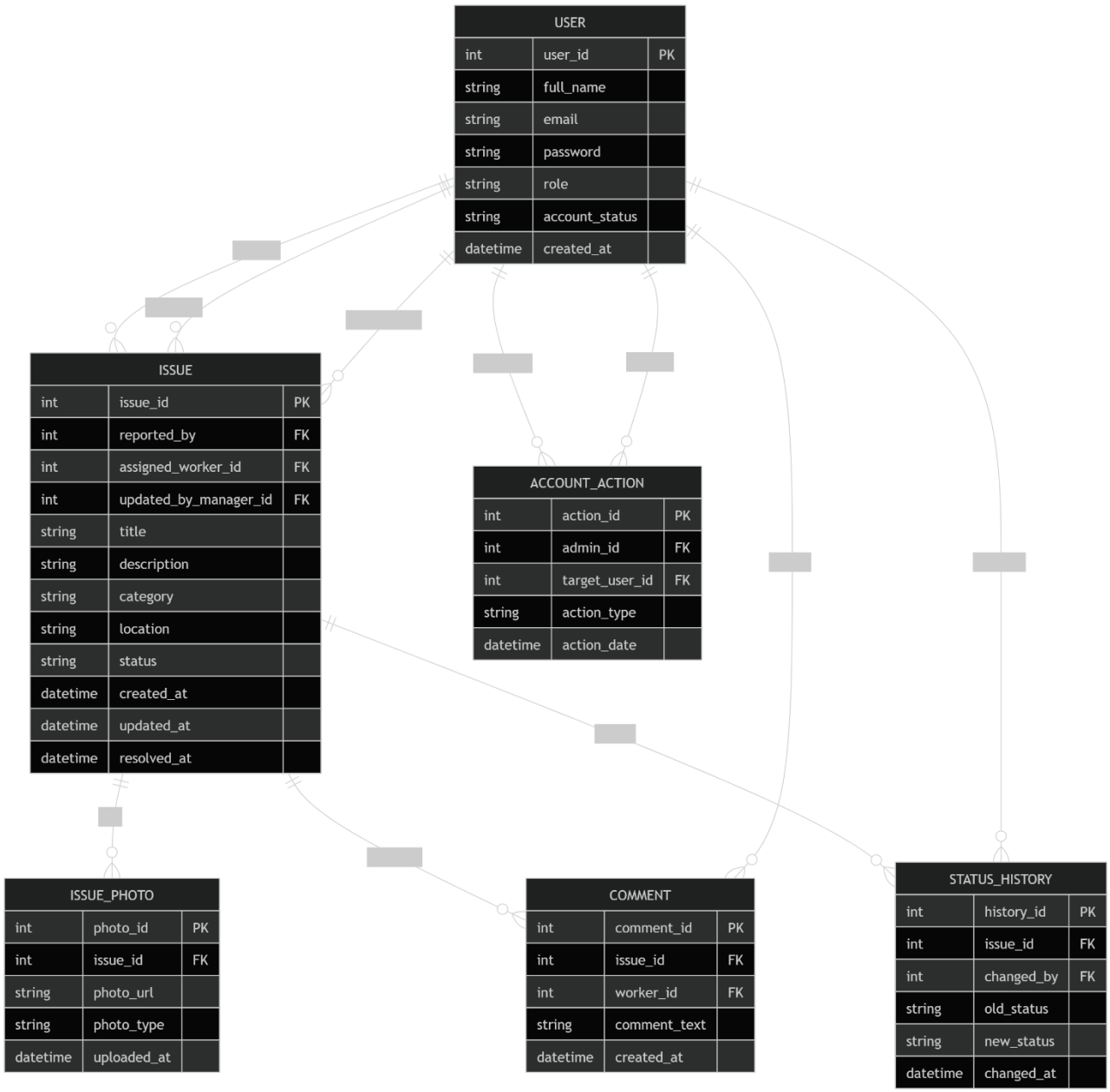
The development and operation of the CampusCare system are subject to several constraints:

- **Platform Constraint:** The system must be developed as a mobile application compatible with both Android and iOS devices.
- **Technology Constraint:** The system will be implemented using React Native for the mobile application and Node.js for the backend services.

- **Time Constraint:** The project must be completed within the academic semester timeline, limiting the scope to essential features only.
- **Resource Constraint:** The system will be developed by a student team with limited resources and experience, which may restrict advanced feature implementation.
- **Network Dependency:** The application requires an internet connection for real-time updates, issue submission, and synchronization.
- **Device Limitation:** The performance of the application may vary depending on the user's device capabilities (camera quality, storage, processing power).
- **Security Constraint:** User authentication and data protection must comply with basic security practices to prevent unauthorized access.
- **Integration Constraint:** The system will operate as a standalone application without integration with existing university systems in the initial version.

.

5. Entity Relation Diagram



6. Technology Stack

The CampusCare system will be developed using the following technologies:

- **Frontend (Mobile Application):** React Native
  - Used to build a cross-platform mobile application for Android and iOS.
- **Backend:** Node.js
  - Handles server-side logic, APIs, authentication, and data processing.
- **Database:** MongoDB
  - Stores user data, issues, assignments, comments, and system records.
- **API Communication:** RESTful APIs
  - Enables communication between the mobile application and backend services.
- **Development Tools:**
  - Visual Studio Code for coding
  - Postman for API testing
  - GitHub for version control and collaboration
- **Design Tools:**
  - Figma (optional) for UI/UX design and prototyping

7. Future Scope of the Project

The CampusCare system can be extended in the future to include additional features that enhance functionality and user experience:

- **Push Notifications:** Real-time notifications for status updates and new assignments.
- **Live Issue Tracking:** Allow users to track issue progress in real time.
- **Rating System:** Enable community members to rate the quality of completed maintenance work.
- **Priority Levels:** Introduce priority classification (e.g., High, Medium, Low) for issues.
- **Analytics Dashboard:** Provide insights and statistics on issue trends, response times, and performance.
- **Integration with University Systems:** Connect with existing campus systems for authentication and facility data.
- **Gamification Features:** Reward users for reporting valid issues to encourage engagement.
- **Offline Mode:** Allow users to submit issues offline and sync them once connected to the internet.
- **AI-Based Predictions:** Predict recurring issues and suggest preventive maintenance actions.